

Цель работы

Изучение метода отбора признаков и подготовленных шагов перед отбором

Формулировка задания

- 1) Реализуйте рассмотренные в лабораторной работе этапы построения и отбора признаков для наборов данных как с использованием Python в соответствующем узле платформы, так и с помощью визуальных инструментов этой платформы.
- 2) Сформулируйте аргументированные заключения о преимуществах или недостатках различных методов анализа.

Ход выполнения задания

В ходе работы был реализован метод отбора признаков и расчет коэффициента корреляции по Пирсону с помощью модуля с программой написанной на Python, и с помощью инструментов Loginom.

Код программы написанной на Python представлен в приложении А.

Общая схема, созданная для выполнения данной работы представлена на рисунке 1.

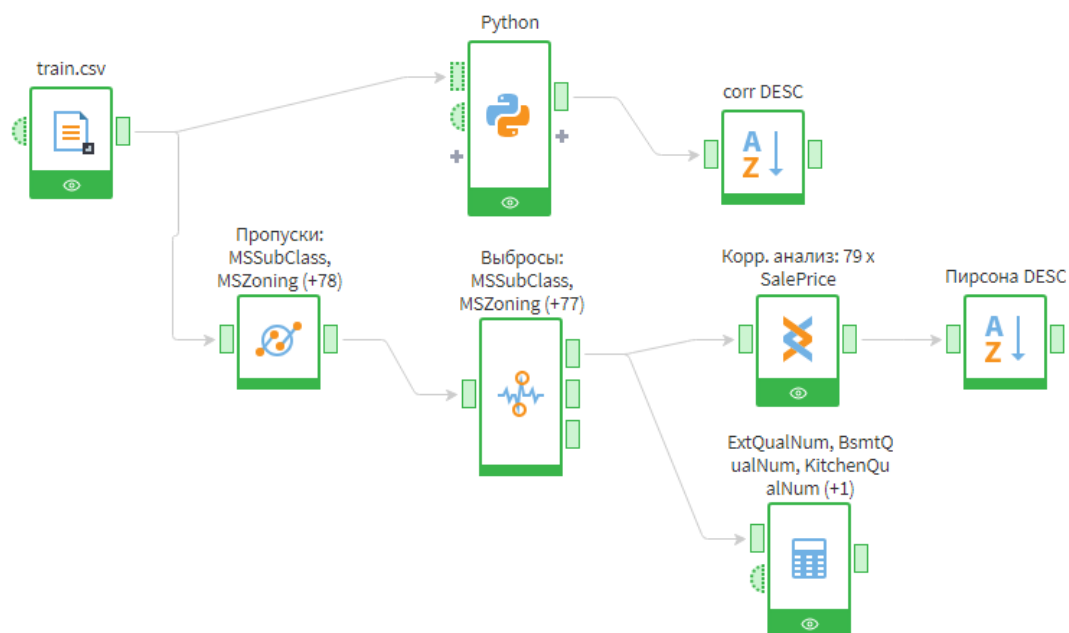


Рисунок 1 – Разработанная схема

Настройки узла заполнения пропусков представлены на рисунке 2.

Заполнение пропусков



Исходные данные упорядочены ☐

Допустимый процент пропусков

60

Random seed

1584668941

№	Входные поля Фильтрация	Вид данных		Метод обработки
1	12 Id	Непрерывный	☐	Не выбран
2	12 MSSubClass	Непрерывный	☒	Заменять медианой
3	ab MSZoning	Дискретный	☒	Заменять наиболее вероятным
4	12 LotFrontage	Непрерывный	☒	Заменять медианой
5	12 LotArea	Непрерывный	☒	Заменять медианой
6	ab Street	Дискретный	☒	Заменять наиболее вероятным
7	ab Alley	Дискретный	☒	Заменять наиболее вероятным
8	ab LotShape	Дискретный	☒	Заменять наиболее вероятным
9	ab LandContour	Дискретный	☒	Заменять наиболее вероятным
10	ab Utilities	Дискретный	☒	Заменять наиболее вероятным
11	ab LotConfig	Дискретный	☒	Заменять наиболее вероятным
12	ab LandSlope	Дискретный	☒	Заменять наиболее вероятным
13	ab Neighborhood	Дискретный	☒	Заменять наиболее вероятным
14	ab Condition1	Дискретный	☒	Заменять наиболее вероятным
15	ab Condition2	Дискретный	☒	Заменять наиболее вероятным
16	ab BldgType	Дискретный	☒	Заменять наиболее вероятным
17	ab HouseStyle	Дискретный	☒	Заменять наиболее вероятным
18	12 OverallQual	Непрерывный	☒	Заменять медианой
19	12 OverallCond	Непрерывный	☒	Заменять медианой
20	12 YearBuilt	Непрерывный	☒	Заменять медианой
21	12 YearRemodAdd	Непрерывный	☒	Заменять медианой
22	ab RoofStyle	Дискретный	☒	Заменять наиболее вероятным
23	ab RoofMatl	Дискретный	☒	Заменять наиболее вероятным
24	ab Exterior1st	Дискретный	☒	Заменять наиболее вероятным
25	ab Exterior2nd	Дискретный	☒	Заменять наиболее вероятным
26	ab MasVnrType	Дискретный	☒	Заменять наиболее вероятным

Рисунок 2 – Настройки узла заполнения пропусков

Настройки узла редактирования выбросов представлены на рисунке 3.

Исходные данные упорядочены



Входные поля		Вид данных
Фильтрация		
☐	12 Id	Непрерывный
☒	12 MSSubClass	Непрерывный
☒	ab MSZoning	Дискретный
☒	12 LotFrontage	Непрерывный
☒	12 LotArea	Непрерывный
☒	ab Street	Дискретный
☒	ab Alley	Дискретный
☒	ab LotShape	Дискретный
☒	ab LandContour	Дискретный
☒	ab Utilities	Дискретный
☒	ab LotConfig	Дискретный
☒	ab LandSlope	Дискретный
☒	ab Neighborhood	Дискретный
☒	ab Condition1	Дискретный
☒	ab Condition2	Дискретный
☒	ab BldgType	Дискретный
☒	ab HouseStyle	Дискретный

Рисунок 3 – Настройки узла заполнения пропусков

Поиск корреляции проводился между всеми полями в качестве признака и ценой в качестве следствия. Настройки узла корреляционного анализа представлена на рисунке 4.

Корреляционный анализ

☒ Коэффициент корреляции Пирсона
 ☐ Коэффициент Тау-б Кендалла

☐ Экстремум взаимнокорреляционной функции
 ☐ Коэффициент корреляции Спирмена

Входные колонки	Набор 1	Набор 2
ab KitchenQual	☒	☐
12 TotRmsAbvGrd	☒	☐
ab Functional	☒	☐
12 Fireplaces	☒	☐
ab FireplaceQu	☒	☐
ab GarageType	☒	☐
12 GarageYrBlt	☒	☐
ab GarageFinish	☒	☐
12 GarageCars	☒	☐
12 GarageArea	☒	☐
ab GarageQual	☒	☐
ab GarageCond	☒	☐
ab PavedDrive	☒	☐
12 WoodDeckSF	☒	☐
12 OpenPorchSF	☒	☐
12 EnclosedPorch	☒	☐
12 3SsnPorch	☒	☐
12 ScreenPorch	☒	☐
12 PoolArea	☒	☐
ab PoolQC	☒	☐
ab Fence	☒	☐
ab MiscFeature	☒	☐
12 MiscVal	☒	☐
12 MoSold	☒	☐
12 YrSold	☒	☐
ab SaleType	☒	☐
ab SaleCondition	☒	☐
12 SalePrice	☐	☒

Рисунок 4 – Настройки узла корреляционного анализа

В результате были получены наборы признак-следствие-коэффициент, которые представлены на рисунках 5, 6, отсортированные по убыванию коэффициента.

Следует отметить, что в программном методе выбирались 20 лучших признаков по модулю, поэтому значения могут отличаться от тех, что были получены с помощью стандартных узлов Loginom.

#	ab Поле1.Имя	ab Поле1.Метка	ab Поле2.Имя	ab Поле2.Метка	9.0 Пирсона
1	OverallQual	OverallQual	SalePrice	SalePrice	0,7919652596
2	GrLivArea	GrLivArea	SalePrice	SalePrice	0,7102850116
3	TotalBsmtSF	TotalBsmtSF	SalePrice	SalePrice	0,6412970276
4	GarageCars	GarageCars	SalePrice	SalePrice	0,6404091973
5	GarageArea	GarageArea	SalePrice	SalePrice	0,6303030007
6	_1stFlrSF	1stFlrSF	SalePrice	SalePrice	0,6219512826
7	FullBath	FullBath	SalePrice	SalePrice	0,5606637627
8	TotRmsAbvGrd	TotRmsAbvGrd	SalePrice	SalePrice	0,5378359088
9	YearBuilt	YearBuilt	SalePrice	SalePrice	0,5237025918
10	GarageYrBltd	GarageYrBltd	SalePrice	SalePrice	0,5204265051
11	YearRemodAdd	YearRemodAdd	SalePrice	SalePrice	0,5071009671
12	Fireplaces	Fireplaces	SalePrice	SalePrice	0,469543199
13	MasVnrArea	MasVnrArea	SalePrice	SalePrice	0,4543538989
14	Foundation	Foundation	SalePrice	SalePrice	0,4452053934
15	BsmtFinSF1	BsmtFinSF1	SalePrice	SalePrice	0,3992993774
16	LotArea	LotArea	SalePrice	SalePrice	0,393468143
17	OpenPorchSF	OpenPorchSF	SalePrice	SalePrice	0,3521685019
18	WoodDeckSF	WoodDeckSF	SalePrice	SalePrice	0,3292461946
19	SaleCondition	SaleCondition	SalePrice	SalePrice	0,3268060712
20	_2ndFlrSF	2ndFlrSF	SalePrice	SalePrice	0,3115464877

Рисунок 5 – Результат отбора признаков с помощью узлов Logitom

#	ab feature	9.0 corr
1	OverallQual	0,7909816006
2	GrLivArea	0,7086244776
3	GarageCars	0,6404091973
4	TotalBathrooms	0,6317310679
5	GarageArea	0,6234314389
6	TotalBsmtSF	0,6135805516
7	TotalSF	0,6135805516
8	FullBath	0,5606637627
9	BsmtQual_Ex	0,553104847
10	TotRmsAbvGrd	0,5337231556
11	YearBuilt	0,5228973329
12	YearRemodAdd	0,5071009671
13	KitchenQual_Ex	0,5040936759
14	Foundation_PConc	0,4977337526
15	MasVnrArea	0,472614499
16	HasFireplace	0,4719080685
17	YearsSinceRemodel	-0,509078738
18	KitchenQual_TA	-0,5192978537
19	AgeAtSale	-0,5233504175
20	ExterQual_TA	-0,5890435234

Рисунок 6 – Результат отбора признаков с помощью узла Python

Результаты выполнения данного анализа обоими способами слегка отличаются друг от друга, что может быть вызвано как неточностью в настройке узлов LogiDom, так и вероятными ошибками при составлении программы. Однако, большинство значений полученных наборов, все таки близки к соответствующим наборам полученным другим способом, что говорит о том, что оба метода выполнены правильно в рамках допустимой погрешности.

Что касается преимуществ и недостатков: Python узел можно настроить куда более точно и идеально подогнать под ситуацию, в то время как LogiDom узлы гораздо проще настраивать. Python узел требует знаний программирования и глубоких знаний о моделировании процесса, в то время как LogiDom узлы не позволяют настроить все с идеальной точностью.

Также, в ходе выполнения работы, был составлен нормализованный график демонстрирующий, как отдельные параметры влияют на параметр, который в исходных считается объединяющим их. График представлен на рисунке 7.

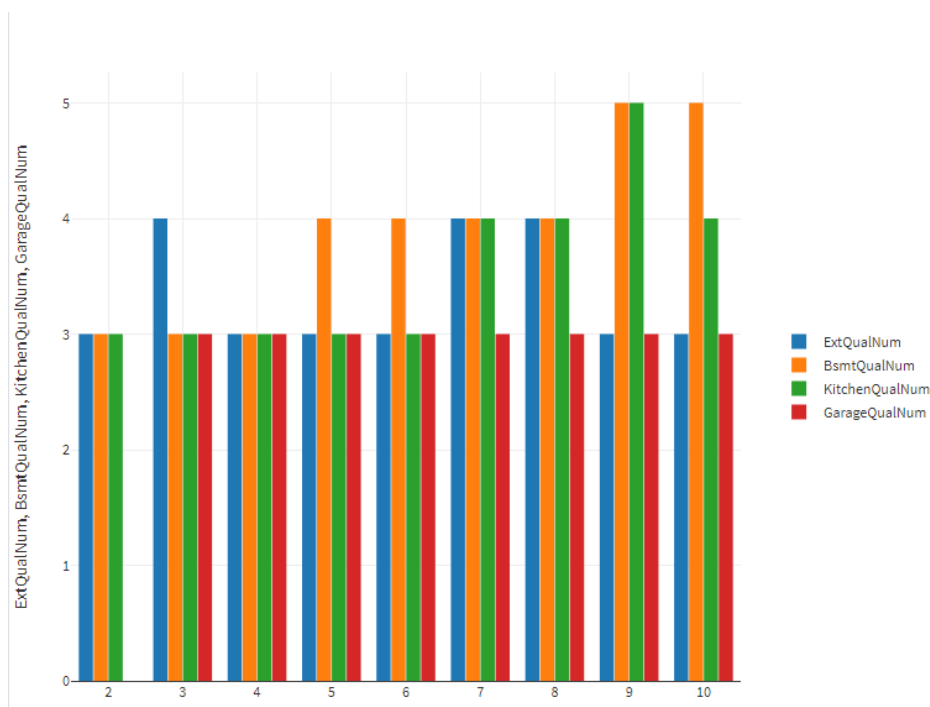


Рисунок 7 – Нормализованный график

ПРИЛОЖЕНИЕ А

Листинг программы

```
from typing import Tuple, Optional, List
import numpy as np
import pandas as pd
import builtin_data
from builtin_data import InputTable, InputTables, InputVariables, OutputTable,
OutputTables, DataType, DataKind, UsageType
from builtin_pandas_utils import to_data_frame, prepare_compatible_table,
fill_table

# =====
# Конфигурация
# =====

CSV_PATH = "train.csv"

# Числовые столбцы по датасету Ames (под ваш CSV)
NUM_COLS = [
    'Id', 'MSSubClass', 'LotFrontage', 'LotArea',
    'OverallQual', 'OverallCond', 'YearBuilt', 'YearRemodAdd',
    'MasVnrArea',
    'BsmtFinSF1', 'BsmtFinSF2', 'BsmtUnfSF', 'TotalBsmtSF',
    '1stFlrSF', '2ndFlrSF', 'LowQualFinSF', 'GrLivArea',
    'BsmtFullBath', 'BsmtHalfBath', 'FullBath', 'HalfBath',
    'BedroomAbvGr', 'KitchenAbvGr',
    'TotRmsAbvGrd',
    'Fireplaces',
    'GarageYrBlt', 'GarageCars', 'GarageArea',
    'WoodDeckSF', 'OpenPorchSF', 'EnclosedPorch', '3SsnPorch', 'ScreenPorch',
    'PoolArea', 'MiscVal', 'MoSold', 'YrSold',
    'SalePrice'
]

# Колонки, где строка 'NA' – валидная категория "отсутствует", а НЕ пропуск:
NA_IS_VALID_CATEGORY = {
    'Alley', 'BsmtQual', 'BsmtCond', 'BsmtExposure', 'BsmtFinType1', 'BsmtFinType2',
    'FireplaceQu', 'GarageType', 'GarageFinish', 'GarageQual', 'GarageCond',
```



```

        'PoolQC', 'Fence', 'MiscFeature'
    }

# Порог для отброса столбцов с очень большим количеством NaN (например, > 60%)
DROP_COL_NA_THRESHOLD = 0.60

# Сколько признаков оставить по корреляции
K_BEST = 20

# =====
# Вспомогательные функции
# =====

def _coerce_numeric(df: pd.DataFrame, num_cols: List[str]) -> pd.DataFrame:
    df = df.copy()
    for c in num_cols:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors='coerce')
    return df

def _treat_na_categories(df: pd.DataFrame) -> pd.DataFrame:
    """
    Для спец-колонок 'NA' оставляем как валидную категорию.
    В остальных object-колонках строковое 'NA' трактуем как пропуск (NaN).
    """
    df = df.copy()
    for col in df.columns:
        if df[col].dtype == object:
            if col in NA_IS_VALID_CATEGORY:
                # Сохраняем 'NA' как валидную категорию, но пустые заменим на
                'NA'

                df[col] = df[col].astype(str)
                df.loc[df[col].isna() | (df[col].str.strip() == '') |
(df[col].str.lower() == 'nan'), col] = 'NA'
            else:
                # В остальных колонках строковое 'NA' -> NaN
                mask_na = df[col].astype(str).str.upper() == 'NA'
                df.loc[mask_na, col] = np.nan
    return df

```

```

def _drop_high_na_columns(df: pd.DataFrame, drop_threshold: float) ->
pd.DataFrame:
    df = df.copy()
    protected = {'SalePrice', 'Id'}
    na_ratios = df.isna().mean()
    to_drop = [c for c, r in na_ratios.items() if (r > drop_threshold and c
not in protected)]
    if to_drop:
        df = df.drop(columns=to_drop)
    return df

def _impute_lot_frontage_by_neighborhood(df: pd.DataFrame) -> pd.DataFrame:
    df = df.copy()
    if 'LotFrontage' not in df.columns:
        return df
    if 'Neighborhood' in df.columns:
        medians = df.groupby('Neighborhood')['LotFrontage'].median()
        def fill_lf(row):
            val = row['LotFrontage']
            if pd.isna(val):
                return medians.get(row['Neighborhood'], np.nan)
            return val
        df['LotFrontage'] = df.apply(fill_lf, axis=1)
    # Общая медиана для оставшихся NaN
    df['LotFrontage'] = df['LotFrontage'].fillna(df['LotFrontage'].median())
    return df

def _s(df: pd.DataFrame, col: str, default):
    """
    Безопасно получить столбец как Series длины len(df).
    Если столбца нет – вернёт Series, заполненную default.
    """
    if col in df.columns:
        return df[col]
    else:
        return pd.Series([default] * len(df), index=df.index)

def _engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """
    Инженерия признаков с безопасными Series-дефолтами.

```

```

"""
df = df.copy()

full = _s(df, 'FullBath', 0)
half = _s(df, 'HalfBath', 0)
bfull = _s(df, 'BsmtFullBath', 0)
bhalf = _s(df, 'BsmtHalfBath', 0)
df['TotalBathrooms'] = full + 0.5*half + bfull + 0.5*bhalf

yrsold = _s(df, 'YrSold', np.nan)
yrbuilt = _s(df, 'YearBuilt', np.nan)
yrremod = _s(df, 'YearRemodAdd', np.nan)
df['AgeAtSale'] = yrsold - yrbuilt
df['YearsSinceRemodel'] = yrsold - yrremod

df['TotalPorchSF'] = _s(df, 'OpenPorchSF', 0) + _s(df, 'EnclosedPorch', 0)
+ \
    _s(df, '3SsnPorch', 0) + _s(df, 'ScreenPorch', 0)

df['TotalSF'] = _s(df, '1stFlrSF', 0) + _s(df, '2ndFlrSF', 0) + _s(df,
'TotalBsmtSF', 0)

df['HasPool'] = (_s(df, 'PoolArea', 0).fillna(0) > 0).astype(int)
df['HasFence'] = (_s(df, 'Fence', 'NA').fillna('NA') != 'NA').astype(int)

gtype = _s(df, 'GarageType', 'NA').fillna('NA')
gcars = _s(df, 'GarageCars', 0).fillna(0)
df['HasGarage'] = ((gtype != 'NA') & (gcars > 0)).astype(int)

df['HasFireplace'] = (_s(df, 'Fireplaces', 0).fillna(0) > 0).astype(int)
df['IsRemodeled'] = (_s(df, 'YearBuilt', -1).fillna(-1) != _s(df,
'YearRemodAdd', -1).fillna(-1)).astype(int)
return df

def _split_columns(df: pd.DataFrame) -> Tuple[List[str], List[str]]:
    real_num = [c for c in NUM_COLS if c in df.columns]
    engineered_numerics =
['TotalBathrooms', 'AgeAtSale', 'YearsSinceRemodel', 'TotalPorchSF', 'TotalSF',

```

```

'HasPool', 'HasFence', 'HasGarage', 'HasFireplace', 'IsRemodeled']
    real_num += [c for c in engineered_numerics if c in df.columns]

# Категориальные – object-тип или из набора NA_IS_VALID_CATEGORY
cats = []
for c in df.columns:
    if c in {'SalePrice', 'Id'}:
        continue
    if (df[c].dtype == object) or (c in NA_IS_VALID_CATEGORY and c not in
real_num):
        cats.append(c)

# Удалим дубликаты
real_num = list(dict.fromkeys(real_num))
return real_num, cats

def _impute_missing(df: pd.DataFrame, num_cols: List[str], cat_cols:
List[str]) -> pd.DataFrame:
    """
    Импутация без sklearn:
    - числовые -> медиана,
    - категориальные -> мода; для спец-колонок, где 'NA' валидно, пустые ->
'NA'.
    """
    df = df.copy()
    for c in num_cols:
        if c in df.columns:
            median = df[c].median()
            df[c] = df[c].fillna(median)
    for c in cat_cols:
        if c in df.columns:
            if c in NA_IS_VALID_CATEGORY:
                df[c] = df[c].fillna('NA')
            # После этого, если остались NaN – заменим на моду
            mode_vals = df[c].mode(dropna=True)
            if not mode_vals.empty:
                df[c] = df[c].fillna(mode_vals.iloc[0])
            else:
                df[c] = df[c].fillna('Unknown')

```

```

return df

def _get_dummies(df: pd.DataFrame, cat_cols: List[str]) -> Tuple[pd.DataFrame,
List[str]]:
    """
    One-Hot кодирование через pandas.get_dummies для заданных категориальных
    колонок.
    """
    df = df.copy()
    dummies = pd.get_dummies(df[cat_cols], prefix=cat_cols, dummy_na=False)
    feature_names = dummies.columns.tolist()
    return dummies, feature_names

def _variance_threshold(X: pd.DataFrame) -> pd.DataFrame:
    """
    Удаляет константные признаки (нулевая дисперсия).
    """
    variances = X.var(axis=0, ddof=0)
    keep = variances[variances > 0].index.tolist()
    return X[keep]

def _select_by_correlation(X: pd.DataFrame, y: pd.Series, k: int) ->
Tuple[pd.DataFrame, List[str]]:
    """
    Выбирает Top-K признаков по модулю корреляции Пирсона с целевой.
    """
    corrs = {}
    y_centered = y - y.mean()
    y_std = y_centered.std(ddof=0)
    for col in X.columns:
        x = X[col]
        # Если дисперсия нулевая – пропускаем
        if x.std(ddof=0) == 0 or y_std == 0:
            continue
        try:
            corr = x.corr(y)
        except Exception:
            corr = np.nan
        if not np.isnan(corr):
            corrs[col] = abs(corr)

```

```

    if not corrs:
        # Если вдруг все NaN – вернем все, как есть
        selected_cols = list(X.columns)
    else:
        sorted_cols = sorted(corrs.items(), key=lambda kv: kv[1],
reverse=True)
        k_eff = min(k, len(sorted_cols))
        selected_cols = [name for name, _ in sorted_cols[:k_eff]]
    return X[selected_cols], selected_cols

# =====
# Основной запуск
# =====

def run(df: Optional[pd.DataFrame] = None
) -> Tuple[pd.DataFrame, np.ndarray, List[str], pd.Series,
pd.DataFrame]:
    """
    Полный цикл без sklearn:
    1) корректная обработка 'NA'-категорий,
    2) приведение числовых типов,
    3) отбрасывание столбцов с > DROP_COL_NA_THRESHOLD пропусков,
    4) специальная импутация LotFrontage по медиане Neighborhood,
    5) построение признаков,
    6) импутация оставшихся пропусков,
    7) One-Hot кодирование (pandas.get_dummies),
    8) отбор признаков по модулю корреляции с SalePrice.

    Возвращает:
    - processed_df: очищенный и обогащённый датафрейм (до One-Hot),
    - selected_X: np.ndarray матрица отобранных признаков,
    - selected_feature_names: список имен отобранных признаков,
    - y: целевая переменная (SalePrice).
    """
    if df is None:
        df = pd.read_csv(CSV_PATH)
    df = df.copy()

    # 1) Обработка 'NA' как категорий/пропусков

```

```

df = _treat_na_categories(df)

# 2) Приведение числовых типов
df = _coerce_numeric(df, [c for c in NUM_COLS if c in df.columns])

# 3) Отброс столбцов с большим числом пропусков
df = _drop_high_na_columns(df, DROP_COL_NA_THRESHOLD)

# 4) Спец-импутация LotFrontage
df = _impute_lot_frontage_by_neighborhood(df)

# 5) Инженерия признаков
df = _engineer_features(df)

# Целевая переменная
if 'SalePrice' not in df.columns:
    raise ValueError("В наборе нет столбца 'SalePrice' – нечего
прогнозировать.")
y = df['SalePrice'].copy()

# 6) Импутация оставшихся пропусков
num_cols, cat_cols = _split_columns(df)
num_cols = [c for c in num_cols if c != 'SalePrice']
df_imp = _impute_missing(df, num_cols, cat_cols)

# 7) One-Hot кодирование категориальных
X_num = df_imp[num_cols].copy()
X_cat, cat_feature_names = _get_dummies(df_imp, cat_cols)
X_all = pd.concat([X_num, X_cat], axis=1)

# Удалим константные признаки
X_all = _variance_threshold(X_all)

# 8) Отбор признаков по корреляции
corr_df_all = _correlation_table(X_all, y)

# если вдруг всё отфильтровалось (редко), то берём все признаки как есть
if corr_df_all.empty:
    selected_feature_names = list(X_all.columns)
else:

```

```

        selected_feature_names = corr_df_all["feature"].head(K_BEST).tolist()

X_selected_df = X_all[selected_feature_names].copy()
selected_X = X_selected_df.values

# Таблица только по отобранным (как вы просили: "Признак - коэффициент
корреляции")
corr_df_selected =
corr_df_all[corr_df_all["feature"].isin(selected_feature_names)].copy()
corr_df_selected =
corr_df_selected.set_index("feature").loc[selected_feature_names].reset_index()
corr_df_selected = corr_df_selected[["feature", "corr"]] # оставим ровно
2 колонки

processed_df = df_imp.copy() # удобен для просмотра и экспорта

# Небольшой лог
print(f"Всего признаков после кодирования: {X_all.shape[1]}")
print(f"Отобрано признаков: {len(selected_feature_names)}")
print("Топ-отобранные признаки (первые 25):")
for n in selected_feature_names[:25]:
    print(" -", n)

return processed_df, selected_X, selected_feature_names, y,
corr_df_selected

def _correlation_table(X: pd.DataFrame, y: pd.Series) -> pd.DataFrame:
    """
    Строит таблицу корреляций Пирсона между каждым столбцом X и y.
    Возвращает DataFrame: feature, corr, abs_corr (отсортирован по abs_corr
desc).
    """
    rows = []
    y_std = (y - y.mean()).std(ddof=0)

    for col in X.columns:
        x = X[col]
        # пропускаем константы / некорректные случаи
        if x.std(ddof=0) == 0 or y_std == 0:
            continue

```



```

try:
    corr = x.corr(y) # Pearson
except Exception:
    corr = np.nan

    if pd.notna(corr):
        rows.append((col, float(corr), abs(float(corr))))

corr_df = pd.DataFrame(rows, columns=["feature", "corr", "abs_corr"])
corr_df = corr_df.sort_values("abs_corr",
ascending=False).reset_index(drop=True)
return corr_df

if __name__ == "__main__":
    df_in = to_data_frame(InputTable)
    processed_df, selected_X, selected_feature_names, y, corr_table =
run(df=df_in)

if isinstance(OutputTable, builtin_data.ConfigurableOutputTableClass):
    prepare_compatible_table(OutputTable, corr_table, with_index=False)
    fill_table(OutputTable, corr_table, with_index=False)

```